

G07 / PROTECTED ANSWERS

G07 REVIEW B

Standalone Zero-to-Job-Ready Data Engineer 2026

Study, practise, check and explain.

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

Read this first 3

Complete coding answers and edge cases 5

A concrete project defense and STAR example 5

Read this first

Open this book after saving your own attempt. Compare the reasoning and result, repair the cause of a mistake, then close the answer and try changed inputs.

Final stage boundary | Master untimed first; optional interview timing only after a correct cold attempt

Protected review: use only after a saved Gate attempt. This is a worked repair guide, not a script to memorize. Repair weak domains, close this file, then take the different fresh gate.

1 | Portfolio truth audit - acceptable answer

Pin the two projects whose README lets a reviewer verify problem, architecture/grain, run route, expected controls, failure/recovery evidence and limitations within about two minutes. Do not pin a flashy project if setup is broken, secrets are exposed, or you cannot defend the design.

2 | SQL + Python hybrid - solved controls

After loading gate_b_events.jsonl, SQL should aggregate purchase rows only. Grain: one row per customer. Expected purchase total=350.00 with C1=100.00, C2=180.00, C3=70.00; only C2 is HIGH_VALUE at threshold >=150.

```
SELECT customer_id, ROUND(SUM(amount),2) AS purchase_amount
FROM events
WHERE type='purchase'
GROUP BY customer_id
ORDER BY customer_id;
flags={customer: ('HIGH_VALUE' if amount>=150 else 'STANDARD')
      for customer,amount in totals.items()}
```

- Reconcile SQL customer totals back to 350.00 before Python classification.
- Python classification must not change the purchase total or drop a customer.

3 | Kafka infrastructure failure

PermissionError on a valid sink write is an infrastructure failure, not poison data. Do not quarantine the valid event as invalid and do not commit/advance its offset. Stop or bounded-retry according to policy, preserve the error evidence, repair the sink, replay the uncommitted event and prove business uniqueness.

4 | <60-second product-event architecture

A defensible design uses keyed ingestion, durable raw/replay, schema contract/evolution, duplicate event identity, PII separation before general analytics, checkpointed stream processing and low-latency serving. Monitor event-time freshness, consumer lag, invalid/quarantine rate and sink errors. State the extra operational/cost complexity compared with batch.

5 | Project defense at 20x volume

Do not say “use a bigger cluster” first. Name the likely bottleneck from your project: input partitioning, shuffle/skew, state growth, sink throughput, small files or orchestration concurrency. Then say what evidence you would inspect and which design change is justified.

6 | Behavioral/English + idempotency teach-back

Use a real example where evidence changed your approach. Plain-English idempotency: “If I safely repeat the same input, the trusted business result does not duplicate or drift.” Name the identity/key/state rule that makes this true in your project.

7 | Application repair

Frequent screens but technical failures mean the top repair is technical interview execution: SQL/Python/troubleshooting/project defense. Choose one measurable weekly experiment such as three cold SQL drills with saved controls and one project defense recording; measure technical-round performance, not raw application count.

8 | Coding - solved shapes

First non-repeating character: frequency map then original-order scan, $O(n)$ time/ $O(n)$ space. ID reconciliation: `set(source)-set(target)` gives missing; `set(target)-set(source)` gives unexpected. State behavior for empty input and duplicates before coding.

Common failure modes

- Treating a sink outage as bad data and committing past it.
- Returning correct purchase totals but changing/dropping rows during Python classification.
- Sub-minute architecture without replay/PII/schema boundaries.
- Application repair that keeps changing the resume when technical rounds are the observed bottleneck.

Repair rubric

- Conceptual prose cannot replace requested runnable SQL/Python evidence.
- Correct numbers without grain and independent controls are partial.
- Architecture without state/replay, quality, failure, security and cost is partial.
- Communication may be simple English; technical truth and structure matter more than accent.
- After repair, close this review and take the different fresh gate without reading these worked solutions.

Complete coding answers and edge cases

```
from collections import Counter

def first_unique(text):
    counts = Counter(text)
    return next((ch for ch in text if counts[ch] == 1), None)

def reconcile_ids(source, target):
    source_set, target_set = set(source), set(target)
    return {
        "missing": sorted(source_set - target_set),
        "unexpected": sorted(target_set - source_set),
    }

assert first_unique("swiss") == "w"
assert first_unique("") is None
assert first_unique("aabb") is None
assert reconcile_ids(["A", "A", "B"], ["A", "C"]) == {
    "missing": ["B"],
    "unexpected": ["C"],
}
```

`first_unique` is case-sensitive and treats spaces as characters. It returns `None` when there is no unique character. `Counter` plus the scan takes $O(n)$ expected time and $O(k)$ space for k distinct characters. ID reconciliation checks set membership, so duplicates do not change membership. If counts also matter, compare `Counter` values separately. Building the sets takes expected $O(n+m)$; sorting the returned differences adds $O(d \log d)$ for d output IDs. Do not claim the sorted implementation is always strictly linear.

A concrete project defense and STAR example

Illustrative practice answer — use it only after reproducing the scenario, and replace it with your own verified experience. “In my local P01 portfolio, the business question was paid order revenue. Day one had nine accepted current orders and paid revenue 2150. A later three-change batch gave eleven current orders and paid revenue 2940. I interrupted publication before the current-result pointer advanced. I checked that the earlier generation stayed visible, then replayed the same input and reconciled the new generation. I used immutable input and a controlled publication boundary to avoid a half-written result. This reference is single-writer and local; I have not demonstrated distributed transactions.”

STAR version: Situation — my local course pipeline could be interrupted during publication. Task — keep readers on a complete result and make replay safe. Action — preserve raw input, stage a new result, inject the failure before publication, inspect the visible generation and replay. Result — only if your own run proves it, report that the old generation remained readable and the rerun produced the expected eleven rows and 2940. Do not turn this into employment or large-scale production experience.

Manager version: “An interrupted update could expose incomplete numbers. I tested an interruption, kept the previous complete report available and made the update repeatable. My checks compare the order count and paid total before I call the update complete.”

For B choose another project and real failure, for example P02's duplicate invoice: explain why aggregation before checking invoice identity could overstate revenue, show the blocked duplicate fixture, then reproduce the valid 1240 total. At 20× volume inspect scan time, partition sizes and join/shuffle evidence before selecting a change. A larger cluster cannot repair a duplicate-invoice contract.